UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES EXTENDIDOS

Sistemas Distribuidos

Clase 09 – Fronteras de arquitectura

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Segundo cuatrimestre 2026

Equipo docente

2026

1 Propósito de la clase

Esta clase cambia la escala del razonamiento. En las clases anteriores estudiamos mecanismos concretos: cómo un motor vuelve durable una decisión, cómo un log conserva una historia, cómo varias réplicas acuerdan un orden y qué ocurre cuando la red se parte. Ahora usaremos esas ideas para decidir *dónde colocar límites dentro de una aplicación completa*.

El caso conductor es **Asteria Online**, un mundo virtual medieval multijugador. No diseñaremos todo el juego. Seguiremos una sola operación: Sofía intenta comprar la pieza única `EspadaSolar#17`. Para completar la compra intervienen Mercado, Inventario, Pagos y, después, Ranking y Notificaciones. La pieza no puede terminar con dos propietarios, el mismo intento no debe cobrarse dos veces y una compra confirmada no puede desaparecer. En cambio, el ranking o una notificación pueden demorarse.

La pregunta central es:

> ¿Qué beneficio concreto obtiene una separación arquitectónica y qué nueva falla, latencia, duplicación de datos o responsabilidad operativa introduce?

El objetivo no es defender monolitos ni microservicios como una moda. El objetivo es aprender a identificar una necesidad observable, comparar una solución simple con una distribuida y exigir evidencia de que el costo adicional de distribuir quedó justificado.

2 Cómo estudiar este capítulo

2.1 Método

Conviene leer cada sección con la misma secuencia:

1. **Necesidad:** ¿qué problema concreto aparece en la compra de la espada?
2. **Intuición:** ¿qué imagen mental permite comprenderlo sin jerga?
3. **Definición:** ¿qué significa técnicamente el concepto?
4. **Ejecución adversarial:** ¿qué ocurre si un mensaje se pierde, un proceso cae o una respuesta llega tarde?
5. **Ventajas y costos:** ¿qué mejora y qué nueva complejidad aparece?
6. **Evidencia:** ¿qué métrica o experimento permitiría revisar la decisión?

No memorice una lista de patrones. Reconstruya la historia: quién posee cada dato, qué promesa se hizo, qué sabe cada participante en cada instante y qué estado debe sobrevivir a una falla.

3 Mapa intuitivo antes del formalismo

Concepto	Idea intuitiva	Pregunta en Asteria
Dominio	ámbito de la realidad con reglas y palabras propias	¿qué significa comprar, reservar o poseer un objeto?
Capacidad	responsabilidad estable que el negocio necesita sostener	¿quién gestiona la propiedad de las piezas?
Frontera	límite explícito entre responsabilidades, reglas y datos	¿qué queda dentro de Inventario y qué queda dentro de Mercado?
Autoridad	componente con la última palabra sobre un hecho	¿quién decide el propietario válido de `#17`?
Responsabilidad	equipo que cambia, opera y recupera esa parte	¿quién atiende una duplicación de objetos?
RPC	solicitud remota que espera una respuesta	¿qué sabe Mercado si Inventario no responde?
Evento	aviso durable sobre un hecho ya confirmado	¿cómo se enteran Ranking y Notificaciones sin bloquear la compra?
Outbox	evento pendiente guardado junto con el cambio de negocio	¿cómo evitar una transferencia sin aviso durable?
Saga	proceso de negocio durable compuesto por varias transacciones locales	¿cómo continuar o reparar reserva, pago y transferencia?
Strangler Fig	migración gradual con camino viejo y nuevo coexistiendo	¿cómo extraer Ranking sin apostar toda la plataforma?

Una arquitectura no es solamente un diagrama de cajas. Es una distribución de autoridad, datos, fallas, comunicación y responsabilidad operativa.

4 El caso conductor y su alcance

Asteria incluye cuentas, mundo, combate, inventario, mercado, pagos, ranking, chat y muchos otros subsistemas. Esta clase se limita al flujo comercial de una pieza única:

1. Sofía pulsa *comprar*.
2. Mercado valida que la oferta siga activa.
3. Inventario reserva `EspadaSolar#17`.
4. Pagos autoriza el cobro.
5. Inventario transfiere el propietario.
6. Ranking y Notificaciones reaccionan después.

Quedan fuera el movimiento en tiempo real, la física, el renderizado, el matchmaking, el anti-cheat y la infraestructura gráfica del cliente. Acotar el caso es importante: una frontera solo puede evaluarse si sabemos qué regla protege y qué operación atraviesa.

4.1 Reglas que no pueden romperse

- `EspadaSolar#17` tiene como máximo un propietario.
- Un mismo intento de compra no se cobra dos veces.
- Una compra confirmada no desaparece después de un reinicio.

4.2 Efectos que pueden demorarse

- actualización del ranking;
- notificación al jugador;
- vista de analítica;
- historial derivado para consulta.

Esta diferencia anticipa una decisión clave: no todo el trabajo pertenece al camino sincrónico de la compra.

5 Vocabulario fundamental

5.1 Dominio, capacidad e invariante

Un **dominio** es una parte de la realidad con problemas, reglas y lenguaje propios. El comercio de objetos incluye ofertas, vendedores, compradores, precios y ventas. El inventario incluye piezas concretas, cantidades, reservas y propietarios.

Una **capacidad** es una responsabilidad estable que el negocio necesita sostener. No es una única función ni un endpoint. *Gestionar la propiedad de objetos* incluye reservar una pieza, transferirla, liberarla, consultar su propietario y evitar dos dueños simultáneos.

Un **invariante** es una regla que debe permanecer verdadera en todo estado válido. En nuestro caso:

$$\#\{\text{propietarios válidos de EspadaSolar\#17}\} \leq 1.$$

Primero se identifica la capacidad y su invariante; recién después se decide si se implementarán dentro de un módulo, proceso o servicio.

5.2 Frontera y dato autoritativo

Una **frontera** es un límite explícito que separa dos ámbitos del sistema. Indica qué responsabilidades, reglas y datos quedan de cada lado y por qué puertas se permite interactuar.

En Asteria, Inventario conserva la propiedad y las reservas; Mercado conserva la oferta, el vendedor y el precio. Mercado no debe modificar directamente la tabla de propietarios. Puede pedir una reserva mediante una operación permitida o reaccionar a un evento confirmado.

El **dato autoritativo** es el registro reconocido como verdad oficial para decidir. Si Inventario dice:

$$\texttt{owner(EspadaSolar\#17)=Sofía},$$

esa es la verdad para transferir o equipar la pieza. Ranking puede mantener una vista derivada, pero no puede decidir el propietario.

5.3 Autoridad y responsabilidad

La **autoridad** es el componente con la última palabra sobre un hecho de negocio. Inventario decide quién posee la espada; Pagos decide si un cobro fue autorizado; Mercado decide el estado de la oferta.

La **responsabilidad** incluye al equipo que puede modificar reglas y datos, aprobar cambios, observar métricas, responder incidentes y recuperar el componente. Autoridad responde *quién decide*; responsabilidad responde *quién sostiene esa decisión durante toda la vida del sistema*.

5.4 Versión, puesta en producción y vuelta atrás

Una **unidad de publicación** es el conjunto de componentes que se actualiza o retira como una misma versión. **Poner en producción** significa hacer que una versión atienda tráfico real. Una **vuelta atrás de versión** o *rollback* retira una versión problemática y devuelve tráfico o software a una versión conocida.

La vuelta atrás de software no deshace pagos, eventos ni transferencias ya confirmados. Los efectos de negocio requieren una compensación, no solamente cambiar binarios.

5.5 Autonomía

La autonomía es el grado en que una parte puede cambiar, publicar una versión, escalar o recuperarse sin obligar al resto a hacer lo mismo al mismo tiempo. Debe observarse mediante preguntas concretas:

- ¿Inventario puede cambiar su lógica sin exigir cambios simultáneos en Mercado?
- ¿Ranking puede poner una versión en producción o volver atrás sin volver a publicar Inventario?
- ¿Ranking puede agregar recursos durante un torneo sin escalar Pagos?
- ¿Una falla de Ranking puede resolverse sin detener la compra?

Separar procesos no crea autonomía automáticamente. Una base compartida, contratos frágiles, versiones coordinadas o un equipo sin capacidad operativa pueden conservar el acoplamiento.

6 De una regla local a un protocolo distribuido

Si una sola autoridad posee el dato, puede proteger el invariante dentro de una transacción local. Inventario puede leer la versión vigente de la pieza, verificar que esté disponible y registrar una reserva dentro de un único commit.

Cuando esa decisión cruza procesos independientes, aparece un **protocolo**: una secuencia acordada de mensajes, estados durables, identificadores, timeouts, reintentos y reglas de recuperación. Mover una frontera no elimina la regla de negocio; transforma una transacción local en un protocolo que debe sobrevivir fallas parciales.

> Distribuir no elimina complejidad: cambia complejidad local por coordinación explícita, observabilidad y recuperación entre procesos.

7 Módulo, proceso, servicio y microservicio

Estos términos no son sinónimos.

Módulo. Frontera de código dentro de una aplicación. Expone una interfaz interna y oculta detalles.

Proceso. Programa en ejecución con memoria, recursos y fallas propias.

Servicio. Capacidad accesible mediante un contrato, normalmente a través de red o mensajería.

Microservicio. Servicio enfocado en una capacidad coherente, con datos y responsabilidad claros, y con independencia razonable de cambio y operación.

Una clase expuesta por HTTP no se convierte por eso en un microservicio. Tampoco todo contexto delimitado necesita un proceso separado.

8 Monolito modular: una opción válida

Un monolito modular se publica como una unidad, pero internamente separa módulos con responsabilidades, interfaces y dependencias explícitas. Compartir un proceso simplifica la comunicación y permite transacciones locales. El problema no es publicar los módulos juntos, sino perder las fronteras internas y permitir que cualquier parte modifique datos ajenos.

La analogía del edificio ayuda si se mapea con precisión:

Analogía	Significado técnico
Edificio	aplicación que se publica como una unidad
Habitación	módulo con responsabilidades definidas
Puerta	API interna permitida
Pared	regla que impide acceder directamente a internals o tablas ajenas
Camino entre edificios	red, serialización, contrato remoto y fallas parciales

8.1 Ventajas

- llamadas locales rápidas y simples;
- transacciones locales para invariantes críticos;
- debugging y pruebas extremo a extremo más accesibles;
- una sola puesta en producción y una sola vuelta atrás;
- menor superficie operativa;
- facilidad para refactorizar fronteras mientras el dominio todavía cambia.

8.2 Riesgos

- dependencias internas sin control;
- acceso directo a tablas de otros módulos;
- cambios que atraviesan muchas responsabilidades;
- tiempos de build y prueba crecientes;

- responsabilidad difusa si todos cambian todo.

Un monolito modular no es una etapa vergonzosa. Puede ser la solución correcta mientras la modularidad interna resuelva el problema.

9 La necesidad concreta que podría justificar separar

La palabra *presión* suele ser ambigua. En esta clase hablaremos de una **necesidad concreta o limitación observable**: algo que impide cumplir una meta del negocio, del rendimiento o de la operación.

Necesidad	Limitación actual	Evidencia
Carga diferente	Ranking consume CPU y afecta la compra	CPU, cola, p99
Ritmos de cambio distintos	Ranking espera la publicación de toda Asteria	tiempo hasta producción
Aislamiento de fallas	un módulo secundario bloquea checkout	alcance del incidente
Seguridad o auditoría	Pagos necesita controles específicos	incidentes y auditorías
Responsabilidad de equipos	todos dependen de todos	tickets, bloqueos, versiones coordinadas

Sin problema, evidencia e hipótesis de mejora, separar solo agrega complejidad. Antes de crear un servicio conviene intentar alternativas más simples: modularidad interna, índices, caché, colas, límites de recursos o un worker separado.

10 Contextos delimitados y lenguaje

Un **contexto delimitado** o *bounded context* es un ámbito en el cual las palabras y reglas mantienen un significado coherente. La palabra *Objeto* significa cosas distintas:

- en Catálogo: tipo de pieza, nombre, rareza y atributos base;
- en Inventario: instancia concreta, propietario, cantidad y versión;
- en Mercado: oferta, vendedor, precio y estado de venta;
- en Combate: arma equipada, daño, durabilidad y tiempo de reutilización.

Forzar un objeto universal produce campos opcionales, permisos confusos, despliegues coordinados y dependencias semánticas ocultas. Es preferible traducir entre modelos mediante contratos explícitos.

Un contexto delimitado no obliga a crear un microservicio. Puede implementarse como módulo dentro del mismo proceso. La frontera semántica debe preceder a la frontera de red.

11 Responsabilidad de datos y acceso global

Database per service no significa necesariamente un servidor físico por servicio. Significa que una autoridad es la única que modifica directamente su modelo. Otros componentes consultan mediante APIs, consumen eventos o mantienen proyecciones.

Para una consulta global existen varias alternativas:

- composición de APIs: más fresca, pero dependiente de varias respuestas;
- vista materializada por eventos: rápida y disponible, pero puede estar atrasada;
- almacén analítico: adecuado para reporting, no para decisiones transaccionales inmediatas;
- índice de búsqueda: optimizado para lectura, reconstruible desde la fuente.

La elección depende de frescura, latencia y disponibilidad; no de la comodidad de hacer un join sobre tablas ajenas.

12 Cadena sincrónica y continuación asíncrona

En una **cadena sincrónica**, la petición espera respuestas antes de continuar:

$$A \to B \to C \to D.$$

Si C tarda o falla, A todavía no puede terminar. Es apropiada cuando el siguiente paso necesita el resultado inmediato, por ejemplo autorizar un pago antes de confirmar una compra.

En una **continuación asíncrona**, un hecho durable permite que otros trabajen después:

$$A \to \text{log} \Rightarrow \{C, D, E\}.$$

A no espera a todos los lectores. Es apropiada para ranking, notificaciones o analítica.

No son estilos universales. Cada interacción debe justificarse por la promesa visible al usuario.

13 El costo adicional de distribuir

Transformar una llamada local en una interacción de red agrega:

- serialización y compatibilidad de formatos;
- latencia y variabilidad de colas;
- timeouts y respuestas perdidas;
- reintentos e idempotencia;
- autenticación, autorización y secretos;
- trazas y correlación;
- puesta en producción y vuelta atrás independientes;
- guardias y procedimientos;
- consistencia explícita entre datos separados.

Este es el **costo adicional de distribuir**. Se acepta si la independencia obtenida es real y medible.

13.1 Disponibilidad compuesta

Como aproximación didáctica, si un flujo necesita cuatro componentes independientes con disponibilidad 0,999 cada uno:

$$A_{flujo} \approx 0{,}999^4 \approx 0{,}996.$$

El cálculo es optimista: supone independencia, ignora red, colas, fallas comunes y degradación. Sirve para recordar que una cadena más larga agrega dependencias, no para predecir exactamente un incidente.

14 Muchos servicios, pero una sola unidad de cambio

La expresión *monolito distribuido* es una etiqueta crítica y deliberadamente paradójica. Físicamente hay varios procesos y red, pero el sistema sigue comportándose como una sola unidad inseparable:

- todos comparten la misma base;
- las versiones deben publicarse juntas;
- una petición atraviesa cadenas RPC largas;
- cambios simples requieren varios equipos;
- una falla local detiene el flujo completo.

Se paga latencia, observabilidad y fallas parciales sin obtener autonomía. En ese escenario, un monolito modular podría ser más simple y ofrecer la misma coordinación.

15 RPC, timeout e idempotencia

Una **RPC** o llamada a procedimiento remoto ofrece un contrato que hace parecer una interacción de red como una llamada a una función. La semejanza sintáctica no elimina la diferencia física.

Supongamos:

1. Mercado envía `Reserve(#17,Sofía)`.
2. Inventario reserva la pieza.
3. La respuesta se pierde.
4. Mercado observa un timeout.

Mercado no puede distinguir entre petición perdida, Inventario lento, reserva ejecutada con respuesta perdida o respuesta todavía en tránsito. Un timeout es una observación local, no un diagnóstico del estado remoto.

Para reintentar con seguridad se necesita una identidad estable, por ejemplo `request_id`. Inventario guarda durablemente la relación entre identidad, efecto y resultado. Ante un reintento devuelve el mismo resultado en vez de reservar otra vez.

16 Comando, evento y consulta

Comando. Solicitud dirigida a una autoridad para que intente cambiar estado: `ReserveItem`.

Evento. Afirmación sobre un hecho ya confirmado: `ItemReserved`.

Consulta. Solicitud de información sin intención de modificar estado: `GetOwner`.

Un evento no debería ser un comando oculto. Si el emisor espera que un único consumidor actúe obligatoriamente, debe modelarse la responsabilidad con claridad.

17 Necesidad y contrato de evento

Después de transferir la pieza, Ranking, Notificaciones y Auditoría deben enterarse sin bloquear la compra. La intuición es simple: un evento es un aviso durable sobre un hecho confirmado.

Ejemplo:

```
OwnershipTransferred(
  event_id, item_id=17,
  from, to=Sofia,
  occurred_at, version
)
```

El **contrato de evento** especifica identidad, significado, esquema, evolución, orden relevante y política de entrega. Se parece a una API porque otros componentes dependen de su interfaz y semántica. La diferencia es que queda almacenado: consumidores nuevos o atrasados pueden leer eventos producidos por versiones anteriores.

Esto aumenta la responsabilidad sobre:

- **identidad:** cómo distinguir eventos y deduplicar;
- **semántica:** qué hecho exacto afirma el evento;
- **evolución:** cómo cambiar el esquema sin romper consumidores históricos.

Las ventajas son desacoplamiento temporal, múltiples lectores, replay y construcción de vistas. Los costos son compatibilidad, duplicados, atraso, orden parcial y observabilidad.

18 Dual write: una intención, dos sistemas

Inventario necesita:

1. actualizar el propietario en su base;
2. publicar `OwnershipTransferred` en un broker.

No existe un único commit que abarque ambas escrituras. Dos órdenes fallan:

18.1 Base primero

$$\text{DB commit} \rightarrow \text{crash} \rightarrow \text{evento ausente.}$$

La propiedad cambió, pero Ranking y Auditoría no se enteran.

18.2 Broker primero

$$\text{evento publicado} \rightarrow \text{rollback/crash de DB.}$$

Se anunció una transferencia que nunca quedó confirmada.

Mover el crash muestra que no existe un orden simple que vuelva atómica la operación.

19 Transactional outbox

19.1 Definición intuitiva

Antes de cerrar el libro contable local, Inventario guarda el cambio de propietario y un aviso pendiente dentro de la misma operación. Si la transacción confirma, ambos existen; si aborta, ninguno existe.

19.2 Definición formal

Una **transactional outbox** es una tabla o colección persistente donde el servicio inserta un evento pendiente dentro de la misma transacción local que modifica el estado de negocio.

```
BEGIN;
UPDATE items SET owner='Sofia' WHERE item_id=17;
INSERT INTO outbox(event_id, aggregate_id,
                   event_type, payload, status)
VALUES (..., 17, 'OwnershipTransferred', ..., 'PENDING');
COMMIT;
```

Un **relay** busca filas pendientes, publica con reintentos y marca progreso. También puede usarse CDC para capturar cambios. Como el relay puede caer después de publicar y antes de marcar la fila, la publicación suele ser al menos una vez.

Un consumidor necesita idempotencia o una **inbox** que recuerde los `event_id` ya procesados.

19.3 Qué resuelve

- evita estado confirmado sin intención durable de publicar;
- permite reanudar tras un crash;
- mantiene la decisión de negocio y el evento pendiente en una transacción local.

19.4 Qué deja abierto

- duplicados del relay;
- atraso entre commit y publicación;
- crecimiento y limpieza de la tabla;
- orden entre distintas entidades;
- consumidores no idempotentes;
- observabilidad y reintentos fallidos.

20 Por qué aparece una saga

La compra atraviesa tres autoridades independientes:

- Mercado decide el estado del intento y la oferta;

- Inventario decide la reserva y propiedad;
- Pagos decide la autorización o rechazo del cobro.

Son independientes porque poseen estados separados, confirman en bases distintas y pueden fallar o reiniciar por separado. No comparten un commit global.

Una **saga** es un proceso de negocio durable compuesto por transacciones locales coordinadas mediante comandos y eventos. Registra dónde quedó el proceso, qué respuesta recibió y qué paso puede ejecutar, reintentar, compensar o escalar a revisión manual.

Estados posibles:

$$\text{CREATED} \to \text{RESERVED} \to \text{PAYMENT_AUTHORIZED} \to \text{OWNERSHIP_TRANSFERRED} \to \text{COMPLETED}.$$

También deben existir estados inciertos:

$$\text{PAYMENT_UNKNOWN}, \quad \text{TRANSFER_PENDING}, \quad \text{COMPENSATING}, \quad \text{MANUAL_REVIEW}.$$

21 Rollback y compensación

Tres mecanismos diferentes suelen confundirse:

Rollback transaccional. Aborta cambios locales que aún no fueron confirmados.

Vuelta atrás de versión. Devuelve tráfico o software a una versión anterior.

Compensación. Ejecuta una nueva acción de negocio para reparar consecuencias ya confirmadas.

Si Pagos autorizó el cobro y luego Inventario no puede transferir, la historia no se borra. La saga puede solicitar reembolso, liberar la reserva, restaurar la oferta y, si no logra terminar, escalar a revisión manual.

Una compensación puede fallar y debe ser idempotente. No es un rollback perfecto.

22 Coreografía y orquestación

La analogía es útil si se vincula con el caso.

22.1 Coreografía

Como en una danza, cada participante conoce las señales y decide su siguiente movimiento. No existe un coordinador central de todo el proceso:

$$\text{ItemReserved} \Rightarrow \text{PaymentAuthorized} \Rightarrow \text{OwnershipTransferred} \Rightarrow \text{PurchaseCompleted}.$$

Ventajas: bajo acoplamiento a un coordinador y autonomía local. Riesgos: flujo difícil de visualizar, ciclos, lógica repartida y compensaciones dispersas.

22.2 Orquestación

Un coordinador durable conserva el estado de la saga y envía comandos a Inventario, Pagos y Mercado. Facilita visualizar estado, timeouts, reintentos, compensaciones y revisión humana. El costo es concentrar conocimiento del proceso y operar otro componente crítico.

No hay una opción universal. Un flujo corto y estable puede coreografiarse; un pago incierto con compensaciones complejas suele beneficiarse de orquestación.

23 Migración gradual: Strangler Fig

El patrón **Strangler Fig** es una migración incremental en la cual el componente viejo y el nuevo coexisten mientras una fachada o router desvía gradualmente operaciones.

La secuencia es:

$$\text{coexistir} \rightarrow \text{comparar} \rightarrow \text{desviar tráfico} \rightarrow \text{retirar lo anterior.}$$

Para Ranking:

1. alimentar el cálculo viejo y el nuevo con los mismos eventos;
2. comparar exactitud, atraso y costo;
3. enviar una fracción de lecturas al nuevo servicio;
4. ampliar solo si las métricas cumplen;
5. conservar la posibilidad de volver el tráfico al camino anterior;
6. retirar el código viejo después de evidencia suficiente.

Una **prueba canary** expone una fracción pequeña de tráfico real a la versión nueva. La vuelta atrás limita el riesgo técnico, pero no deshace efectos de negocio ya confirmados.

24 Preparación operativa

Separar implica sostener otro sistema. La **preparación operativa** o *readiness* es evidencia de que el equipo puede publicar, observar, recuperar y mantener la capacidad en producción.

24.1 CI y CD

CI o integración continua integra cambios con frecuencia y ejecuta compilación, pruebas y controles automáticos.

Entrega continua mantiene cada cambio validado listo para producción, aunque la promoción pueda ser manual. **Despliegue continuo** pone automáticamente en producción todo cambio que supera los controles. La sigla CD es ambigua y debe aclararse.

24.2 Guardia, procedimiento, SLI y SLO

Una **guardia** es la rotación responsable de recibir alertas y coordinar incidentes. Un **runbook** o procedimiento contiene pasos para diagnosticar, mitigar y recuperar una falla repetible.

Un **SLI** es un indicador medido, por ejemplo latencia p99 o porcentaje de compras correctas. Un **SLO** es una meta cuantificada para ese indicador durante una ventana, por ejemplo 99,9% de compras correctas por mes.

Las trazas distribuidas correlacionan el recorrido de una petición o evento a través de procesos.

24.3 Checklist mínima

- CI y publicación de versiones;
- vuelta atrás probada;
- permisos y secretos;
- logs, métricas y trazas;
- timeouts, reintentos e idempotencia;
- SLO y alertas;
- guardia y procedimientos;
- backup y recuperación;
- pruebas de falla;
- capacidad y costos.

25 Métricas que demuestran autonomía

Sin una línea base anterior no puede demostrarse que separar mejoró el sistema.

Métrica	Pregunta que responde
Tiempo hasta producción	¿la frontera acelera el paso desde cambio listo hasta versión funcionando?
Publicaciones sin coordinación	¿los equipos pueden publicar sin esperar a otros?
Escala independiente	¿se agregan recursos solo donde crece la carga?
Alcance del incidente	¿cuántos usuarios, datos y capacidades afecta una falla?
p99 y errores	¿la cola lenta y la tasa de fallas cumplen el SLO?
Coordinaciones entre equipos	¿disminuyeron tickets, aprobaciones y bloqueos?

Si el nuevo servicio sigue compartiendo base, versiones, guardias y procedimientos con la aplicación, la independencia prometida no se materializó.

26 Método reusable para decidir una frontera

Una propuesta madura responde cinco grupos de preguntas:

1. **Problema y evidencia:** ¿qué necesidad persiste y cómo se mide?
2. **Regla, dato y autoridad:** ¿qué invariante contiene la frontera y quién decide?
3. **Alternativa simple:** ¿por qué no alcanza mejorar la modularidad interna?
4. **Costo, falla y mecanismo:** ¿qué agrega la red y cómo se recupera?
5. **Métrica y revisión:** ¿qué resultado justificaría continuar o volver atrás?

Este método se conecta con el marco del curso:

$$D = (P, F, G, S, T),$$

con problema, fallas, garantías, solución y trade-offs declarados antes de elegir tecnología.

27 Errores frecuentes y cómo corregirlos

Error	Corrección
"Monolito es código desordenado"	separar forma de puesta en producción de calidad modular
"HTTP convierte una clase en microservicio"	exigir capacidad, dato, autoridad y responsabilidad operativa
"Más servicios aumentan disponibilidad"	dibujar la cadena sincrónica y mover una falla
"Evento significa exactamente una vez"	separar entrega, identidad y efecto idempotente
"Outbox publica una sola vez"	mover el crash después de publicar y antes de marcar progreso
"Saga es una transacción distribuida"	mostrar estados intermedios y compensaciones falibles
"Rollback borra el pasado"	distinguir versión de software, rollback local y compensación
"Separar crea autonomía"	medir publicaciones independientes, base compartida y guardias

28 Bibliografía guiada

- **DDIA, capítulo 1:** confiabilidad, escalabilidad y mantenibilidad. Lea pensando qué evidencia justifica distribuir.
- **DDIA, capítulo 4:** REST, RPC y dataflow por mensajes. Observe evolución independiente de contratos.
- **DDIA, capítulo 7:** transacciones. Compare lo que protege una transacción local con una saga.
- **DDIA, capítulo 11:** CDC, event sourcing y procesamiento de streams. Relacione con outbox, replay e idempotencia.
- **DDIA, capítulo 12:** sistemas derivados y dataflow. Aplique a Ranking como vista reconstruible.
- **Evans:** contextos delimitados y lenguaje ubicuo.
- **Fowler:** Monolith First, trade-offs de microservicios y Strangler Fig.
- **Richardson/AWS:** transactional outbox, sagas y migración incremental.

29 Trabajo Final Integrador

Para el Trabajo Final Integrador, cada equipo deberá justificar las fronteras de su sistema. No alcanza con dibujar servicios. Para una capacidad elegida, documente:

1. dominio, capacidad e invariante;
2. dato autoritativo y responsable técnico-operativo;
3. problema observable que motiva la separación;
4. alternativa modular más simple y razón para descartarla;
5. interacción sincrónica y asíncrona;
6. timeout, reintento, identidad e idempotencia;
7. dual write y estrategia outbox si corresponde;
8. saga, estados inciertos y compensaciones;
9. estrategia de migración y vuelta atrás;
10. SLI, SLO y métricas de autonomía;
11. prueba de falla;
12. condición para revisar o revertir la decisión.

30 Glosario operativo

Autoridad. Componente con la última palabra sobre un hecho.

Autonomía. Independencia medible de cambio, publicación, escala o recuperación.

Canary. Exposición gradual de tráfico a una versión nueva.

Capacidad. Responsabilidad estable del negocio que agrupa reglas, datos y operaciones.

Comando. Solicitud para que una autoridad intente cambiar estado.

Compensación. Nueva acción que repara un efecto ya confirmado.

Contexto delimitado. Ámbito donde lenguaje y reglas son coherentes.

Contrato de evento. Identidad, significado, esquema y evolución de un hecho publicado.

Dato autoritativo. Registro reconocido como verdad oficial.

Dual write. Necesidad de escribir dos sistemas sin una transacción común.

Evento. Hecho confirmado que puede ser consumido posteriormente.

Frontera. Límite explícito entre responsabilidades, reglas y datos.

Guardia. Rotación que responde alertas e incidentes.

Idempotencia. Propiedad por la cual repetir una intención no multiplica su efecto lógico.

Inbox. Registro local de eventos ya procesados.

Outbox. Evento pendiente guardado con el cambio de negocio.

Relay. Proceso que publica filas pendientes de la outbox.

RPC. Llamada remota que espera una respuesta.

Saga. Proceso durable compuesto por transacciones locales y compensaciones.

SLI/SLO. Indicador medido y meta cuantificada para ese indicador.

Strangler Fig. Migración gradual con camino viejo y nuevo coexistiendo.

Vuelta atrás. Retorno de tráfico o software a una versión anterior.

31 Autoevaluación

Responda sin mirar las diapositivas:

1. ¿Qué diferencia existe entre una frontera semántica y una frontera de red?
2. ¿Por qué una capacidad no es una única operación?
3. ¿Qué sabe Mercado después de un timeout de Inventario?
4. ¿Qué problema resuelve outbox y qué problemas permanecen?
5. ¿Por qué una compensación no equivale a rollback?
6. ¿Dónde está la autoridad sobre la propiedad de `#17`?
7. ¿Qué ventajas y riesgos tiene coreografiar la compra?
8. ¿Qué métrica demostraría autonomía de Ranking?
9. ¿Qué condición haría preferible mantener Inventario dentro del monolito modular?
10. ¿Cómo migraría Ranking de forma reversible?

32 Síntesis final

La arquitectura de Asteria no mejora por multiplicar procesos. Mejora cuando una frontera resuelve una necesidad persistente, conserva una regla clara, asigna autoridad y responsabilidad, explicita las fallas de comunicación y produce evidencia de una mejora real.

El diseño unido es preferible mientras la modularidad interna permita evolucionar, escalar y recuperar el sistema con un costo razonable. Distribuir se vuelve justificable cuando una necesidad concreta no puede resolverse de manera suficiente dentro de esa unidad y el equipo está preparado para operar la complejidad nueva.

> Una frontera vale cuando resuelve un problema medible, protege su invariante y el equipo puede sostener las fallas que introduce. Si las métricas no mejoran, la decisión debe revisarse.